ownership of the thread. This avoids any unpleasant consequences should the `thread_guard` object outlive the thread it was referencing, and it also means that no one else can join or detach the thread once ownership has been transferred into the object. Because this would primarily be aimed at ensuring threads are completed before a scope is exited, I named this class `scoped_thread`. The implementation is shown in the following listing, along with a simple example.

Listing 2.6. `scoped_thread` and example usage

```
class scoped_thread
{
    std::thread t;
public:
    explicit scoped_thread(std::thread t_):                    ❶
        t(std::move(t_))
    {
        if(!t.joinable())                                      ❷
            throw std::logic_error("No thread");
    }
    ~scoped_thread()
    {
        t.join();                                              ❸
    }
    scoped_thread(scoped_thread const&)=delete;
    scoped_thread& operator=(scoped_thread const&)=delete;
};

struct func;                                                   See listing 2.1

void f()
{
    int some_local_state;
    scoped_thread t(std::thread(func(some_local_state)));      ❹

    do_something_in_current_thread();
}                                                              ❺
```

The example is similar to that from listing 2.3, but the new thread is passed in directly to the `scoped_thread` ❹ rather than having to create a separate named variable for it. When the initial thread reaches the end of `f` ❺, the `scoped_thread` object is destroyed and then joins with ❸ the thread supplied to the constructor ❶. Whereas with the `thread_guard`